

P3384R1

COUNTER

Published Proposal, 2025-03-21

This version:

<https://isocpp.org/files/papers/P3384R1.html>

Author:

[Jeremy Rifkin](#)

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

<https://github.com/jeremy-rifkin/proposals/blob/main/cpp/counter.bs>

Abstract

`COUNTER` is a predefined macro provided as a language extension by all major C and C++ implementations. This paper aims to standardize existing practices and match WG14's adoption of `COUNTER` for C2Y.

Table of Contents

1	Revision History
1.1	R1
2	Introduction
3	Previous Proposals
4	Rationale for Standardization

5 Motivating Examples

6 Implementation Support

7 Design Considerations

7.1 Precompiled Headers

7.2 Modules

7.3 ODR

7.3.1 Possible Solutions

7.3.2 Proposed Solution

7.3.2.1 Is `__COUNTER__` still needed?

7.4 Evaluation Order and Caching

7.5 Argument Handling

7.6 Literal Form

7.7 Library Usage

7.8 Range and Overflow

8 Proposed Wording

References

Normative References

Informative References

§ 1. Revision History

§ 1.1. R1

- Updated to reflect `__COUNTER__` has been accepted by WG14
- Updated wording to match [\[N3457\]](#)

§ 2. Introduction

The `__COUNTER__` predefined macro is a common language extension for C and C++ which expands to an integer literal that starts at `0` and increments by `1` every time it is expanded in a translation unit.

This is useful for generating unique identifiers, generating unique indices, and other preprocessor metaprogramming uses.

§ 3. Previous Proposals

`__COUNTER__` has not been proposed to WG21, however, it was briefly mentioned previously in a WG14 paper, *Extensions to the preprocessor for C2Y* [N3190]. The meeting minutes, [N3227], mention brief discussion of `__COUNTER__`'s implications on evaluation order and caching. No polls were taken on [N3190].

[N3457], which provides a focused proposal on `__COUNTER__` for C, has been accepted by WG14 into C2Y. Given the shared preprocessor between C and C++, this paper follows accepted WG14 wording.

This paper provides a focused proposal on `__COUNTER__` and aims to provide additional context and motivation.

§ 4. Rationale for Standardization

`__COUNTER__` is de-facto portable today. Every major implementation supports it with unsurprising semantics. However, there is inherent uncertainty surrounding its portability and semantics due to it not being standardized.

Codebases striving for maximum portability must resort to detection and fallback such as this example from [google benchmark](#):

```
// Check that __COUNTER__ is defined and that __COUNTER__ increases by 1
// every time it is expanded. X + 1 == X + 0 is used in case X is defined
// empty. If X is empty the expression becomes (+1 == +0).
#if defined(__COUNTER__) && (__COUNTER__ + 1 == __COUNTER__ + 0)
#define BENCHMARK_PRIVATE_UNIQUE_ID __COUNTER__
#else
#define BENCHMARK_PRIVATE_UNIQUE_ID __LINE__
#endif
```

Meanwhile other C++ codebases avoid the macro altogether due to this uncertainty. In the absence of cautious checking and fallback, a developer must consult numerous widely used C++ implementations to convince themselves that `__COUNTER__` exists and does what they want.

In the case of google benchmark, `__LINE__` is an adequate fallback due to how `BENCHMARK` macros are typically used. However, this is not an adequate general-purpose replacement due to it not being unique in the general case.

While every major C++ compiler today supports `__COUNTER__`, it's not always enabled. For example, EDG only provides it outside of standards mode.

Additionally, minor divergences in `__COUNTER__` semantics are observable (see [§7.5 Argument Handling](#)), though they do not impact most use cases.

Due to fairly widespread use, both in C and C++, it would be useful to incorporate the existing practice of `__COUNTER__` into the official standard in order to provide more clear portability and semantic guarantees.

§ 5. Motivating Examples

A brief survey of some uses of `__COUNTER__` in the C and C++ community:

C++:

- Google benchmark uses `__COUNTER__` for [unique identifiers](#), falling back to `__LINE__` if `__COUNTER__` isn't present or doesn't behave as expected
- Google Orbit uses `__COUNTER__` for [unique identifiers](#)
- LLVM uses `__COUNTER__` for [unique identifiers](#) as well as in sanitizer code to [prevent ICF](#)
- Catch2 uses `__COUNTER__` for [unique identifiers](#), falling back to `__LINE__`
- Tensorflow uses `__COUNTER__` extensively, primarily for [unique identifiers](#)
- Chromium uses `__COUNTER__` for unique identifier generation, e.g. in [crash logging code](#), as well as for creating [unique tags for `ABORT\(\)`s](#)
- Folly uses `__COUNTER__` for [unique identifiers](#), falling back to `__LINE__` if not present
- v8 uses `__COUNTER__` for [unique identifiers](#)

C:

- Linux uses `__COUNTER__` for, among other things:
 - Unique identifiers in instrumentation; [link](#)
 - Preventing duplicate inline assembly blocks; [link](#)
 - Unique identifiers in [initcalls](#)

- Systemd uses `__COUNTER__` for unique identifiers in multiple places [link 1](#) [link 2](#)
- Netdata uses `__COUNTER__` for [unique identifiers](#)
- The QMK firmware project uses `__COUNTER__` auto-incrementing endpoint numbers; [link](#)
- Yabai, a macOS window manager, uses `__COUNTER__` for unique indices in timing utilities; [link](#)
- Drgn, a programmable debugger, uses `__COUNTER__` for unique identifiers throughout the code-base; [link](#)
- Metric Panda Games uses `__COUNTER__` for [lookup tables](#) as part of a localization and compile-time string hashing system.

Many additional uses include use for static assertions, however, that use case is now covered by built-in static assertion facilities.

§ 6. Implementation Support

`__COUNTER__` has long been supported by all major implementations of C and C++:

Compiler	Earliest Version On Compiler Explorer
GCC	3.4.6  (earliest version supporting <code>__COUNTER__</code> : 4.4.7 )
Clang	3.0.0 
MSVC	19.0 
ICC	13.0.1 
ICX	2021.1.2 
EDG	6.5 

 : Supported only outside standards mode, requiring either microsoft, GCC, or Clang emulation mode to be enabled (controlled with `--microsoft`, `--g++`, and `--clang` respectively).

Comparison: <https://godbolt.org/z/fqTs9sWx6>

Additionally, C compiler support excluding duplicates from above:

Compiler	Earliest Version On Compiler Explorer
cc65	2.17 
Chibicc	2020-12-07 

Compiler	Earliest Version On Compiler Explorer
CompCert	3.9 ✓
Cproc	Trunk ✓
EDG	6.5 🟡
Movfuscator	Trunk ✓
ppci	0.5.5 ✗
SDCC	4.0.0 ✓
TCC	0.9.27 ✓
TenDRA	Trunk ✗
z88dk	2.2 ✓
Zig cc	0.6.0 ✓

Comparison: <https://godbolt.org/z/Mx4MznMaY>

§ 7. Design Considerations

§ 7.1. Precompiled Headers

MSVC and GCC save the state of `__COUNTER__` in precompiled headers. GCC notes that the `__COUNTER__` macro must not be expanded prior to inclusion of a pre-compiled header. If it is, then the precompiled header is not used.

This paper proposes no requirements for `__COUNTER__` behavior surrounding pre-compiled headers.

§ 7.2. Modules

GCC and MSVC do not propagate `__COUNTER__` across modules, including for header units. The following compiles with a linker error due to multiple definitions of `x0`:

```
// header.hpp
#define CONCAT_IMPL(x, y) x##y
#define CONCAT(x, y) CONCAT_IMPL(x, y)
#define NEW_VAR(name) CONCAT(name, __COUNTER__)
```

```
int NEW_VAR(x); // x0
int NEW_VAR(x); // x1

// main.cpp
import "header.hpp"
int NEW_VAR(x); // x0
```

There are similar concerns with `__TIME__` and `__DATE__` macros surrounding header units, though the potential for problems is less pronounced. One option would be to disallow the expansion of `__COUNTER__` in header units, however, no such restriction is proposed in this paper.

This paper proposes no change to the current behavior. Other behaviors would introduce additional complexity without clear benefit.

§ 7.3. ODR

It's possible to inadvertently violate ODR with `__COUNTER__`:

```
// foo.hpp
#define CONCAT_IMPL(x, y) x##y
#define CONCAT(x, y) CONCAT_IMPL(x, y)
#define NEW_VAR(name) CONCAT(name, __COUNTER__)
inline void foo() {
    int NEW_VAR(x) = 2;
}

// a.cpp
#include "foo.hpp"

// b.cpp
int x = __COUNTER__;
#include "foo.hpp"
```

Current implementations do not make any special attempt to diagnose or prevent such use of `__COUNTER__` beyond existing ODR diagnostics. Similar ODR issues can occur as a result of `__DATE__` and `__TIME__`. While existing practice is that these ODR issues exist, it is worthwhile looking at possible solutions to the problem.

N.b.: Similar considerations exist for C, however, different `inline` function definitions can only result in unspecified behavior.

§ 7.3.1. Possible Solutions

This is a difficult problem to solve due to the nature of `__COUNTER__` and how the preprocessor interacts with the rest of the language. Possible solutions include:

- Just don't use `__COUNTER__` in `inline` functions in headers
- Provide a mechanism to reset the `__COUNTER__`, or even push and pop the counter
- Allow for multiple counters `__COUNTER__`, possibly tied to `__FILE__`
- Change ODR to allow `inline void foo() { int x0; }` and `inline void foo() { int x1; }` to not be ill-formed
- Some sort of deterministic `__UUID__` or `__UNIQUE__` macro that is tied to the file and line

Most of these would not be practical, would add boilerplate, or would introduce substantial complexity.

§ 7.3.2. Proposed Solution

This paper proposes no fundamental changes to existing `__COUNTER__` functionality or language semantics. Instead, unique identifiers for variables in header-`inline` functions should be solved by:

1. Modules, where `__COUNTER__` is module-local
2. The `_` placeholder [P2169], which is ODR-friendly

This proposal does not preclude additional functionality or other approaches to make `__COUNTER__` more ODR-friendly at a later time.

§ 7.3.2.1. Is `__COUNTER__` still needed?

`_` is largely sufficient for uses of `__COUNTER__` in the case of local identifiers, however, it does not cover use-cases of `__COUNTER__` in namespace-scoped identifiers or other preprocessor metaprogramming uses.

As an example of use of `__COUNTER__` beyond local identifiers google benchmark uses uniquely-named identifiers at namespace-scope to register benchmark functions:

```
// after preprocessor expansion:
static ::benchmark::internal::Benchmark* _benchmark_2FooBar __attribute__((
```



```

    ::benchmark::internal::RegisterBenchmarkInternal(
        new ::benchmark::internal::FunctionBenchmark("FooBar", FooBar)
    )
);

```

An alternative to `__COUNTER__` in cases such as this would be to standardize `__attribute__((constructor))`. Google benchmark does not rely on `_benchmark_2FooBar` to manage any objects, it is a pure constructor. However, in cases where an object is managed and possibly needs to be destructed at the end of a program, using a namespace-scoped variable consolidates the constructor and destructor logic around an object, rather than managing the object between free functions and a variable. I.e.:

```

std::optional<Foo> obj;
__attribute__((constructor)) void obj_setup() {
    obj = setup_foo();
}
/* possibly a destructor too */

```

vs:

```

Foo obj = setup_foo();
/* or some raii-wrapper around Foo if additional destruction logic is needed

```

While `__` covers many uses of `__COUNTER__`, the preprocessor utility continues to be useful due to existing practice, uses outside local identifiers, other preprocessor metaprogramming uses of `__COUNTER__` beyond unique identifiers. Additionally, because of interest in avoiding divergence in the shared preprocessor, its use in C is an important consideration.

§ 7.4. Evaluation Order and Caching

Meeting notes for [N3190] mention brief discussion of `__COUNTER__` making evaluation order and caching observable, possibly leading to divergence between compiler test cases [N3227].

The evaluation order consideration is similar to existing expression evaluation order observability. Experience from existing practice has shown this doesn't pose a substantial concern or footgun. This is especially true given the typical use for `__COUNTER__`, which is unique identifiers/indices. When it

comes to caching, a conforming implementation would need to take care to not cache macro expansions involving `__COUNTER__`.

This paper proposes no change to the current behavior.

§ 7.5. Argument Handling

If passed as an argument to a function-like macro, the argument is expanded exactly once and then preprocessing tokens naming the parameter are each replaced with the resulting token sequence. This paper proposes the following case should produce `0 0`. This is the current behavior on all compilers tested on Compiler Explorer except Chibicc, which produces `0 1`.

```
#define X(Z) Z Z
X(__COUNTER__) // 0 0
```

In the case of an unused argument, `__COUNTER__` will not be expanded and will not result in an increment. This is the current behavior for all compilers tested on Compiler Explorer.

```
#define F00(X)
F00(__COUNTER__)
__COUNTER__ // 0
```

Additionally in the case of `__VA_OPT__` but no use of `__VA_ARGS__`, an expansion of `__COUNTER__` and subsequent increment should occur. Currently, Clang diverges from MSVC and GCC in the following example. It produces `0` while the others produce `1`:

```
#define X(...) __VA_OPT__()
X(__COUNTER__)
__COUNTER__ // 1
```

Notably, Clang produces the desired output in the following example:

```
#define X(...) __VA_OPT__(a)
X(__COUNTER__)
__COUNTER__ // 1
```

The proposed wording does not address this `__VA_OPT__` corner case and it remains an area of implementation divergence.

If used by the stringizing operator or token pasting operator, `__COUNTER__` should not be incremented. This is the current behavior on all compilers tested on compiler explorer.

```
#define STR(X) #X
STR(__COUNTER__) // "__COUNTER__"
#define CONCAT(X) A##X
CONCAT(__COUNTER__) // A__COUNTER__
__COUNTER__ // 0
#define CONCAT2(X) A##X X
CONCAT2(__COUNTER__) // A__COUNTER__ 1
__COUNTER__ // 2
```

§ 7.6. Literal Form

Matching [N3457], the expansion of `__COUNTER__` produces an integer literal with no digit separators whose value is representable as `long int`. Digit separators are excluded so that `##` concatenation with a preceding identifier always produces a valid identifier token. [N3457] recommends that implementations ensure all expansions produce literals of the same type (e.g. by appending the suffix `L`) or diagnose when the type changes.

§ 7.7. Library Usage

Standard library headers or macros defined by them may internally expand `__COUNTER__`. Whether such expansions occur is unspecified. This matches [N3457]’s wording.

§ 7.8. Range and Overflow

`__COUNTER__` is implemented with an `unsigned` counter in GCC and Clang and both implementations wrap around to zero when that counter overflows. This paper specifies that the number of expansions of `__COUNTER__` in a translation unit shall not exceed 2147483647, otherwise the program is ill-formed. This limit aligns with the `__LINE__` / `#line` directive limit of 2147483647 per `[cpp.line/3]` and matches [N3457].

§ 8. Proposed Wording

Proposed wording relative to [\[N4950\]](#):

Insert a bullet point in [\[cpp.predefined/1\]](#) before bullet 3:

- COUNTER
The expansion of COUNTER produces an *integer-literal* not containing *digit separators*. The produced *integer-literal* is such that concatenation with a preceding *identifier* token using the `##` operator (`[cpp.concat]`) results in an *identifier* token, and such that when interpreted the value is representable as `long int`. The value of the first expansion in a translation unit is `0`, and each subsequent expansion produces a value one greater than the previous.

It is unspecified whether the inclusion of standard library headers or invocation of macros defined by them results in expansions of COUNTER.

The number of expansions of COUNTER in a translation unit shall not exceed `2147483647`.

Recommended practice: Implementations should either diagnose when an expansion of COUNTER produces a literal of a different type than the previous expansion, or ensure all resulting integer literals have the same type, for example by including a literal suffix.

Update [\[cpp.predefined/3\]](#):

The values of the predefined macros (except for `__FILE__` and `__LINE__`, and COUNTER) remain constant throughout the translation unit.

Update the second bullet in [\[cpp.subst/1\]](#):

- Otherwise, the replacement preprocessing tokens are the preprocessing tokens of corresponding argument after all macros contained therein have been expanded, where for each such parameter this expansion is performed exactly once and then preprocessing tokens naming the parameter are each replaced with the resulting token list. The argument's preprocessing tokens are completely macro replaced before being substituted as if they formed the rest of the preprocessing file with no other preprocessing tokens being available.

§ References

§ Normative References

[N4950]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: <https://wg21.link/n4950>

§ Informative References

[N3190]

Extensions to the preprocessor for C2Y. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3190.htm>

[N3227]

Draft Minutes for 22–26 January, 2024. URL: <https://www9.open-std.org/JTC1/SC22/WG14/www/docs/n3227.htm>

[N3457]

COUNTER. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3457.htm>

[P2169]

A nice placeholder with no name. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2169r4.pdf>